

Title Slide

- Project: FintechHQ
- AI-Enabled Financial Intelligence, Analytics, and Paper Trading Platform
- 2026 presentation edition

Project Vision

- Build a unified fintech platform for secure access, market research, AI analytics, and paper trading.
- Reduce fragmentation between charting, fundamentals, news, alerts, learning, and strategy testing.
- Provide an extensible academic and engineering foundation rather than a single-purpose demo.

System Architecture

- Next.js 16 frontend with React 19, TypeScript, Tailwind CSS, Radix UI primitives, Recharts, Zustand, Framer Motion, and lightweight-charts for interactive market visualization.
- FastAPI backend that exposes versioned REST APIs for authentication, user management, stocks, strategies, trading, alerts, community, and education workflows.
- Dedicated FastAPI ML microservice that handles feature engineering, price prediction, recommendation, risk analytics, fundamental analysis, backtesting, sentiment scoring, and AI chart-pattern detection.
- PostgreSQL with TimescaleDB-style time-series usage for market and OHLCV storage, supported by Alembic migrations and SQLAlchemy ORM models.
- Redis for caching, fast state lookups, and service-side infrastructure support.
- Docker Compose orchestration for frontend, backend, ML service, database, Redis, MLflow, and Flower.

Technology Stack

- Frontend: Next.js 16 app router, React 19 and TypeScript, Tailwind CSS 4, Radix UI and shadcn-style components, Recharts and lightweight-charts, NextAuth scaffold and local token storage flow, Framer Motion and tsparticles for interactive UI polish
- Backend: FastAPI, SQLAlchemy 2 async ORM, Alembic migrations, PostgreSQL / TimescaleDB usage pattern, Redis, python-jose for JWT handling, passlib with Argon2, sha256_crypt, and bcrypt support, slowapi for rate limiting, yfinance, pandas, numpy, ta
- ML Service: FastAPI ML microservice, PyTorch, scikit-learn, transformers, MLflow, OpenCV headless, Pillow, mplfinance, SciPy and custom algorithmic signal processing
- DevOps: Docker Compose, Dockerfiles per service, MLflow tracking container, Flower monitoring container, Container networking and mounted development volumes

Authentication and Security

- Access and refresh JWT tokens with explicit token type claims.
- Argon2 password hashing with compatibility fallbacks for legacy hash migration.
- Password policy requiring length, mixed case, numerics, and special characters.
- Breached-password screening using HaveIBeenPwned k-anonymity API logic.
- Failed-login tracking and temporary account lockout after repeated failures.
- User session recording with device fingerprint and IP address capture.

Frontend Modules

- Auth screens for login, register, password reset, and 2FA setup.
- Dashboard, portfolio, technical analysis, fundamentals, news, trading, strategies, backtest, risk, alerts, community, and learning routes.
- Reusable UI widgets including charts, ticker search, password strength meter, order book, and dashboard cards.

Backend Modules

- Versioned routers for auth, users, stocks, trading, strategies, alerts, social, and education.
- Service layer for alerts, broker simulation, stock data, education, and social flows.
- WebSocket layer for price, portfolio, and alert channels.

ML and AI Modules

- Feature engineering, risk analytics, LSTM forecasting, recommendation engine, and sentiment analysis.
- Fundamental scoring and DCF-style valuation support.
- Algorithmic plus CNN-transformer chart-pattern detection with annotation pipeline.

Detailed Data Model

- User: Identity, account status, risk profile, verification state, lockouts, timestamps, and security relationships.
- PasswordHistory: Tracks previous password hashes to block reuse.
- UserSession: Stores per-device session tokens, expiry, fingerprint, and IP metadata.
- Stock / OHLCVData / FundamentalData: Market master data, historical candles, and stored fundamentals.
- Portfolio / Position / Transaction: Cash balance, holdings, and transaction ledger for paper trading.
- Order / Trade: Orders, execution status, fills, and trading activity.
- Strategy / BacktestRun: User-defined strategies and simulation results.
- Alert / Notification / AlertHistory: Threshold definitions, user alerts, read-state tracking, and trigger logs.
- Post / Comment / Like / Follow: Community interaction graph and social feed data.

- Module / Lesson / UserProgress / Enrollment / Quiz / QuizSubmission: Learning management and progress tracking.

Backend API Catalog

10

- Watchlist / WatchlistItem: User-defined monitoring lists.
- MLPrediction / NewsArticle / NewsSentiment: Predictive outputs and enriched news records.
- /api/v1/auth/register: Registers a user after uniqueness checks, password policy validation, breach screening, and password-history initialization.
- /api/v1/auth/login/access-token: Authenticates users, enforces lockouts, supports 2FA branching, and issues access plus refresh tokens.
- /api/v1/auth/login/verify-2fa: Completes a TOTP-based login challenge and opens a real session.
- /api/v1/auth/refresh: Refreshes access tokens using refresh-token semantics.
- /api/v1/auth/2fa/setup: Generates a TOTP secret and provisioning URI.
- /api/v1/auth/2fa/verify: Finalizes 2FA setup.
- /api/v1/users/me: Returns the authenticated user profile.
- /api/v1/users/forgot-password: Generates a reset token with demo-mode response behavior.
- /api/v1/users/reset-password: Validates reset token, enforces password rules, blocks reuse, and invalidates sessions.

- /api/v1/stocks/search: Searches stored stocks and supplements results from the curated Indian stock universe.

Extended Backend API Catalog

11

- /api/v1/stocks/market-movers: Returns gainers and losers using live or fallback data.
- /api/v1/stocks/index-stocks: Supplies Nifty, Sensex, or combined symbol catalogs.
- /api/v1/stocks/{ticker}/quote: Supplies latest quote data.
- /api/v1/stocks/{ticker}: Returns or hydrates stock master information.
- /api/v1/stocks/{ticker}/fundamentals: Returns fundamentals and valuation-oriented data.
- /api/v1/stocks/{ticker}/ohlcv: Provides candle data from DB or live provider fallback.
- /api/v1/stocks/{ticker}/news: Fetches and sentiment-enriches asset-level news.
- /api/v1/stocks/{ticker}/indicators: Calculates SMA, EMA, RSI, MACD, and Bollinger Bands server-side.
- /api/v1/trading/orders: Places or lists paper-trading orders.
- /api/v1/trading/orders/{order_id}: Cancels pending paper orders.
- /api/v1/trading/positions: Returns holdings enriched with simulated live prices.
- /api/v1/trading/portfolio/summary: Provides aggregate portfolio values.
- /api/v1/alerts: Creates, lists, and deletes alerts.
- /api/v1/alerts/notifications: Lists and marks notifications as read.
- /api/v1/strategies: Lists, creates, and deletes strategy definitions.
- /api/v1/social/feed: Returns social feed content.
- /api/v1/social/posts: Creates community posts.

- /api/v1/social/posts/{id}/comments: Adds comments to community posts.

- /api/v1/education/seed: Seeds learning content.

ML API Catalog

- /api/v1/education/modules: Lists modules and learning state.

- /api/v1/education/lessons/{id}/complete: Marks lessons complete, calculates support/resistance levels.

- /api/v1/training/train: Runs model-training related workflows.

- /api/v1/sentiment/predict: Scores text sentiment for financial context.

- /api/v1/prediction/train: Trains an LSTM-based price model.

- /api/v1/prediction/forecast: Forecasts the next price from recent history.

- /api/v1/recommendation/analyze: Produces a recommendation from price behavior and optional news text.

- /api/v1/technical/analyze: Runs technical-analysis logic for chart-based insight.

- /api/v1/risk/analyze: Calculates portfolio risk metrics from holdings and history.

- /api/v1/fundamental/analyze: Computes DCF and health-score outputs.

- /api/v1/backtest/run: Executes a simplified backtesting routine.

- `/api/v1/pattern-detection/detect-from-data`: Runs ensemble pattern detection and optionally returns annotated charts.

Implementation Honesty

- `/api/v1/pattern-detection/detect-image`: Placeholder image-based path pending trained vision weights.
- The codebase contains a mix of production-style backend services and UI screens that are either API-integrated or still backed by representative mock data.
- `/api/v1/pattern-detection/patterns-history`: Returns stored detection history.
- `/api/v1/pattern-detection/supported-patterns`: Lists supported classical chart patterns.
- Authentication, password reset, stock search, quotes, OHLCV retrieval, indicator calculation, alerts, community posting, education seeding, and several ML endpoints are implemented as concrete service flows.
- Some screens such as portfolio, trade, risk, and backtest currently render realistic demo data in the frontend while the backend and ML layers already expose foundations for fuller integration.
- The project uses legacy naming in places such as 'FinTech AI Platform' and older repository documents; this document set standardizes the academic/project title as FintechHQ, as requested.
- Several UI pages still render mock or staged data even though backend service foundations are present.
- Reset-password flow exposes a reset token in the response for demo convenience and should not do so in production.

Academic Takeaways

- The project demonstrates domain modeling, full-stack architecture, security engineering, and AI integration in one codebase.
- It exposes realistic engineering trade-offs between live features and staged UI scaffolds.
- It is well suited for capstone reporting, professional documentation, and future commercialization paths.

User Workflows

- User onboarding: A user registers with strong credentials, optionally configures 2FA, receives tokens, and enters the dashboard.
- Market research: The user searches Indian symbols, loads OHLCV data, studies indicators, checks quotes, reads news, and explores fundamentals.
- Technical analysis: The technical-analysis screen fetches candles, indicator overlays, quotes, and ML-backed chart-pattern findings.
- Paper trading: Trading logic validates buying power, updates positions, records transactions, and maintains order state.

Current Maturity

- The codebase contains a mix of production-style backend services and UI screens that are either API-integrated or still backed by representative mock data.
- Authentication, password reset, stock search, quotes, OHLCV retrieval, indicator calculation, alerts, community posting, education seeding, and several ML endpoints are implemented as concrete service flows.
- Some screens such as portfolio, trade, risk, and backtest currently render realistic demo data in the frontend while the backend and ML layers already expose foundations for fuller integration.
- The project uses legacy naming in places such as 'FinTech AI Platform' and older repository documents; this document set standardizes the academic/project title as FintechHQ, as requested.

Limitations and Risks

- Several UI pages still render mock or staged data even though backend service foundations are present.
- Reset-password flow exposes a reset token in the response for demo convenience and should not do so in production.
- The 2FA setup page currently uses mocked frontend data rather than invoking the real backend setup endpoint.
- The backend configuration defines an RS256 label, while token creation and decode helpers currently use HS256 symmetric signing internally.
- Real broker integration is not implemented; trading is a paper-trading simulation.
- Pattern image detection depends on trained YOLO or related weights and currently documents that limitation in the API response.

Future Scope

- Live broker integration and fully wired frontend modules.
- Improved model lifecycle, experiment tracking, and validation.
- Production-grade secrets, observability, and cloud deployment.

Conclusion

- FintechHQ already demonstrates strong breadth across fintech, AI, and security engineering.
- Its modular design supports future growth without major architectural replacement.
- The project is suitable for academic presentation, viva, and technical review.